# Technical AI Engineer Test – Shopee

**Gregorius Bayu Aji Wibisono**

## Engineering Knowledge AI Agent Test

1. *Describe differences between REST API, MCP in the context of AI.*

   REST API usually has more general purpose (can be used for anything), with the main purpose is only to connect any two programs. Simple and general, but because of its simplicity, the connection will break if API changes, and that changes happened almost every time a program gets an update. The worst part is, in this economy, AI has an update almost every month. So, if the AI developer wants to add new feature or the AI needs another output/input format, they must create another endpoint, and every client of the application must update its code to use it.

   On the other hand, MCP already designed for LLMs. It's more dynamic and adaptable to all the changes on the AI development world. It also has a uniform protocol for all servers. So, if the AI developer wants to create more features, they only need to add a new tool to the server. They don't need to create another "connection door", because with MCP, clients can automatically discover the new tool and use it. Makes it easier for clients to integrate their programs with an AI.

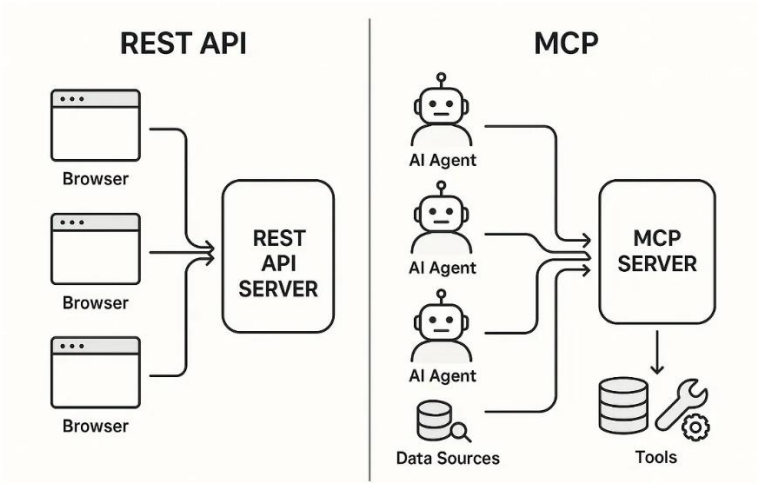



Figure 1. REST API vs MCP Difference
Source: Medium

2. *How REST API, MCP, can improve the AI use case?*

   At the first of its release, AI (for example, ChatGPT) only can be used by using website access. It cannot be used by any applications. That made it hard for people to use them. But later, REST API was released. It can connect AI to the other systems/applications that are used by humans (not just use website access), which made AI more usable for usual daily problems

   But as the time goes by, AI applications needed to connect to more and more tools and data sources. Every AI application had to write custom code for every tool, which made it hard for developers to maintain. So, Anthropic created MCP, an open protocol that standardized how AI applications connect to tools and

backend systems. That makes the client not need to update its code every time a new tool is added. It just needs to connect to the MCP Server once, and the AI can discover and use the new tools automatically.

3. *How do you ensure that your AI agent answers correctly?*

I ensure my AI Agent answers correctly by testing it before I release it. First, I prepare a Ground Truth. Ground Truth is a real document samples with correct answers that are verified manually. Ground Truth became my truth based that controls the result of LLM. The samples include both clean and dirty document. Then I measure the accuracy for each field. Even though the result won't be 100% (if I reach 100%, usually I'll check it and called it an overfitting), the test lets measure my AI Agent so I can know its performance. That makes me able to monitor it and fix its weaknesses. When the agent gives a wrong answer, I check it manually against the ground truth. I run the test repeatedly until I can ensure my agent will give a good answer to every user.

4. *Describe what can you do with Docker / Containerize environment in the context of AI*

By using Docker, I can package the AI feature with all its dependencies (python code, libraries, configurations) into an image that allows me to run the same environment on any machine. This avoids configuration problems, version differences, and other problems. It also makes me able to pack the same identical training & testing (or even production) environment.

This image can also be shared with other people, so they can run it on their machines/laptops, makes it easier for other employee to study and continue our work, and reduces onboarding/training time. Even so, since each image has a version, I can also update or roll back easily. Docker can also isolate code that ran by an AI agent. That means, the main system will only sends the code and input to the container, so every mistake won't affect the main system.

5. *How do you finetune the LLM model from raw?*

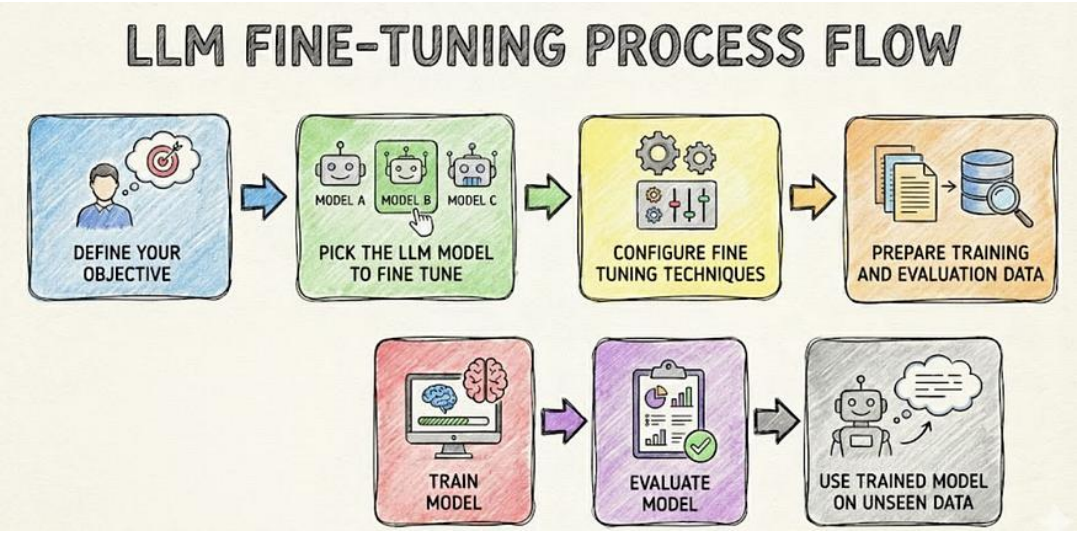



Figure 1. LLM Fine-Tuning Process Flow

Source: Medium

First, after we define our objective. Usually, the hard part for people is deciding between fine-tuning, prompt engineering, and RAG. Fine-tuning is usually done when we need the model to learn specific behaviour, like a consistent output format or a specialized task, that's hard to achieve if we only just do prompt. But if we need the model to only know new facts, RAG is usually better.

After that, we need to choose the right LLM model to be fine-tuned (because not every model support fine-tuning), considering whether it's open-weight, whether its size fits our hardware, and other considers.

The third step is we need to choose the fine-tuning method. There are usually two main fine-tuning techniques; Full fine-tuning, when we update all parameters of the LLM, and Parameter-Efficient Fine-Tuning (such as LoRA or QLoRA), when we update only a small subset of the model's parameters.

When we already define the objective, configure the steps, and pick which LLM model that are going to be fine-tuned, we prepare the data. Here, we don't just pick the raw data. We prepare the data. We clean it by removing duplicates, broken data, or ineffective information. Then, we convert it into instruction-answer pairs in JSONL format, following the model's chat template, and we split it into training, validation, and testing data.

After we cleaned the data, we can train the pre-trained model with our data using libraries like Hugging Face TRL or Unsloth, on a GPU such as Google Colab (or on our own laptop if we have GPU). Before that, we also need to configure training parameters such as learning rate, batch size, epoch numbers, and others. During training, we need to monitor the validation loss to avoid overfitting. Then, after training, we evaluate the fine-tuned model on the test set and compare it with the original model to make sure it actually improved and didn't lose its general abilities.